

Entity Query Language

Syntax, Semantics, and Complexity

Anonymous Author(s)

October 8, 2026

Contents

1	Overview	2
1.1	Motivation and Design Principles	2
1.2	Relation to Existing Query Languages	2
2	Vocabulary and the Knowledge Base	3
2.1	Signature	3
2.2	The Knowledge Base	4
2.3	Variable Symbols	5
3	Syntax	5
3.1	Terms	5
3.2	Atomic Formulas	6
3.3	Formulas (Query Bodies)	6
3.4	Queries	6
3.5	Result Quantifiers	7
3.6	Rules	7
4	Illustrative Example	7
5	Semantics	9
5.1	Domain Closure and Unique Names	9
5.2	Evaluation Semantics	9
5.3	Answer Set	10
5.4	Aggregation Semantics	10
5.5	Negation as Failure	10
5.6	Semantics of Terms	10
5.7	Semantics of Atoms	11
6	Reasoning on Assignment	11
6.1	Property Facts and Rules	11
6.2	The Assignment Procedure	12
6.3	Soundness and Closure at Runtime	12
7	The OWL 2 RL/RDF Rules in Ontomatic	13
8	Computational Complexity	20

1 Overview

Entity Query Language (EQL) is a symbolic, declarative, relational query language with a First Order Logic like syntax that is embedded natively in Python, designed to make Python object-oriented domain models directly queryable without any additional representation layer. It separates the symbolic representation of a query from its evaluation, so that the same query can be evaluated by different backends: in memory, translated to SQL over a relational database, or by a probabilistic backend.

This document specifies the default backend, which evaluates queries in memory under closed-world reasoning: domain closure and negation as failure.

This document provides a formal specification of EQL’s syntax, semantics, and computational complexity.

1.1 Motivation and Design Principles

Existing query languages impose a representational gap between the domain model and the query layer. SQL operates over flat relational tables and requires either manual schema maintenance or an Object-Relational Mapping (ORM) layer to bridge the gap to object-oriented code. SPARQL operates over RDF triples, requiring a separate knowledge graph representation. Datalog and its extensions operate over symbolic facts that must be populated from the application separately. In all cases, the developer must maintain two representations of the same domain: one in their programming language, one in the query system.

EQL eliminates this gap. Its design rests on the following principles:

1. **No impedance mismatch.** EQL operates directly on live Python objects. Querying working memory needs no schema, no tables and no object-relational mapping; ORMatic generates these from the same classes only when the knowledge is also kept in a database. The object-oriented domain model the developer already writes *is* the knowledge base.
2. **Executable queries as first-class Python.** Queries are Python expressions, not strings passed to an external compiler. This means queries are statically analysable by development tools, debuggable at runtime, and subject to normal Python error handling and type checking. There is no compilation boundary between query and application code.
3. **General programs as query components.** Any Python callable—including arbitrary programs—can participate in a query as a predicate or symbolic function, subject to determinism and purity requirements. This goes beyond the built-in predicates of SQL or the fixed vocabulary of SPARQL.
4. **Structural depth.** EQL queries traverse nested objects, lists, and dictionaries natively. There is no restriction to flat relational structure. An attribute of a field that is itself a list of objects is queried by index or method access, not by a join.
5. **Axioms in class definitions.** A class can state its axioms as class methods that return an EQL condition characterizing its members. The axioms live in the class definition alongside the data model, can be inspected, and can be used as constraints or as the bodies of rules.

1.2 Relation to Existing Query Languages

EQL’s logical foundations are drawn from conjunctive query evaluation over finite relational structures [3, 2], Negation as Failure [4], and the TBox/ABox distinction of Description Logics [1]. It differs from these traditions in three key respects.

SQL and relational databases: SQL requires a separate schema, table-based storage, and an ORM layer to map objects to relations. EQL queries the object model directly; attribute chains replace most joins, and the remaining ones are expressed by shared variables and membership atoms.

Datalog: Datalog rules are stored in the knowledge base and applied to a fixpoint. In EQL, rules are user-invoked programs; there is no fixpoint. EQL’s fact base is always a flat finite set of ground object instances. Even though rules exist they are not automatically invoked when answering queries unless they are called as part of the query.

SPARQL and RDF: SPARQL requires a triple-store representation distinct from the application model. EQL operates on the application’s own Python objects with no translation step.

Contributions. This document makes the following formal contributions:

1. A typed signature $\Sigma = (\mathcal{S}, \mathcal{C}, \mathcal{M}, \mathcal{P}, \mathcal{F})$ for EQL with a formal TBox/ABox knowledge base structure and a subsumption ordering on class symbols.
2. A distinction between query variables $\mathcal{V}_Q$ (a class annotation and a finite domain, given or taken from the objects in memory) and inference variables $\mathcal{V}_I$ (class-annotated, domain populated from rule body answer sets), formalising EQL’s rule mechanism.
3. An abstract syntax covering terms with structural depth (field, index, and method access), three atom forms including Boolean term atoms $[t]_{\mathbb{B}}$, body formulas, conjunctive queries, unions, result quantifiers, and rules.
4. A model-theoretic semantics in the single closed-world interpretation of the knowledge base (Reiter’s closed-world assumption), with negation as failure, and the γ operator of relational algebra for aggregation.
5. Computational complexity bounds for progressively richer EQL query classes, distinguishing data complexity from combined complexity.

2 Vocabulary and the Knowledge Base

2.1 Signature

A *signature* $\Sigma = (\mathcal{S}, \mathcal{C}, \mathcal{M}, \mathcal{P}, \mathcal{F})$ consists of non-logical symbols over which the EQL language is built. Variable symbols $\mathcal{V}$ are logical symbols introduced separately in Section 2.3.

Class symbols. A finite set of *class symbols* $\mathcal{S} = \{S_1, S_2, \dots, S_k\}$ corresponds to Python classes. The class hierarchy defined by the developer induces a *subsumption ordering* $\sqsubseteq$ on $\mathcal{S}$, where $S' \sqsubseteq S$ if and only if S' is a subclass of S in Python’s class system (i.e., `issubclass(S', S)` holds).

Constant symbols. A finite set of *constant symbols* $\mathcal{C}$ denotes the Python object instances populating the domain. Each constant is typed by one or more classes via the class hierarchy.

Mapped symbols. A finite set of *mapped symbols* $\mathcal{M}$. Each $m \in \mathcal{M}$ is a named attribute of a class $S \in \mathcal{S}$, representing a property that is intrinsic to S and accessible on any instance of S via dot notation. They are referred to as Mapped symbols because they can be seen as a mapping applied to S . The attribute m may be:

- a *field*, typed $m : S \rightarrow T$, returning a value of type T .

- an *indexable field*, typed $m : S \rightarrow \text{Indexable}[K, T]$, supporting element access by key $k \in K$, where K is any hashable key type (e.g., $\mathbb{Z}$ (i.e., integers) for lists and tuples, or an arbitrary hashable type for dictionaries).
- a *method*, typed $m : S \times T_1 \times \cdots \times T_n \rightarrow T$, invocable with arguments.

How these attributes are accessed syntactically is defined in the term grammar (Section 3).

Predicate symbols. A finite set of *predicate class symbols* $\mathcal{P}$. Each $p \in \mathcal{P}$ is a Python class that is a subclass of `Predicate` and implements a characteristic function via `__call__(self) -> bool`. Predicates occupy two roles:

1. *Constraint role* (query body): the expression $p(\bar{t})$ is an atomic formula whose truth value is determined at evaluation time by Boolean-casting the ground predicate instance (Section 5.7).
2. *Domain role* (fact base): instances of p are ordinary Python objects and may be stored in $\mathcal{A}$, queried over, and used as arguments in other atoms.

Function symbols. A finite set of *symbolic functions* $\mathcal{F}$. Each $f \in \mathcal{F}$ is a Python callable of one of the following forms:

1. A Python function decorated with `@symbolic_function`.
2. A class method accessed through an EQL variable.
3. A subclass of `SymbolicFunction`, the class form of a symbolic function.

Evaluation stores no function outputs. An object that user code constructs is an ordinary object, however: instances of classes whose instances are tracked in memory (by default, the subclasses of `Symbol` other than roles and predicates) are registered there on construction, where variables without a given domain find them.

Every $f \in \mathcal{F}$ and every characteristic function $p \in \mathcal{P}$, and every other piece of user code that evaluation invokes (property getters and overridden comparison, hashing and iteration methods), is assumed **pure** (no side effects on $\mathcal{A}$ or shared state) and **deterministic** (same inputs always produce the same output), and **terminating** on every input that evaluation passes to it. These conditions are required at all times to ensure correctness and termination of query evaluation. An acyclic call graph over $\mathcal{F} \cup \mathcal{P}$ rules out unbounded mutual recursion between callables, but it does not imply termination, since a single non-recursive callable may loop. These are semantic preconditions (Section 5.2); they are not statically checkable for arbitrary Python callables [10].

2.2 The Knowledge Base

Definition 2.1 (EQL Knowledge Base). An EQL *knowledge base* is a pair $\mathcal{K} = (\mathcal{T}, \mathcal{A})$ where:

- $\mathcal{T}$ (*TBox*) is the Python class hierarchy: the set of class symbols $\mathcal{S}$, their attribute symbols $\mathcal{M}$, and the subsumption ordering $\sqsubseteq$, together with the classes' axioms and the predicates and functions. The property descriptors of KRROOD, which apply property rules when a value is assigned, are abstracted away here: their consequences are already part of the attribute values. Section 6 formalizes how descriptors compute them.
- $\mathcal{A}$ (*ABox*) is a finite set of ground typed object instances. $\mathcal{A}$ contains no disjunctive assertions, no conditional facts, and no evaluable statements.

2.3 Variable Symbols

Let $\mathcal{V} = \mathcal{V}_Q \cup \mathcal{V}_I$ be the set of *variable symbols*. These are the logical symbols of EQL, ranging over domains of constants drawn from $\mathcal{C}$. EQL provides two types of variables:

Query variables $\mathcal{V}_Q$: each carries a class annotation $S \in \mathcal{S}$ and a finite *domain* D , written `variable( $S$ , domain= $D$ )`; only the elements of D that are instances of S are admissible. If D is omitted, it is the set of instances of S in memory, for classes whose instances are tracked; for other classes a domain must be given. `variable_from( $D$ )` creates a variable from a domain alone. A domain may also depend on other variables: `flat_variable( $s$ .takes_course)` ranges over the elements of the attribute `takes_course` of the value of s . We write $\llbracket D_x \rrbracket_\theta$ for the domain of x under an assignment θ of the variables it depends on. Query variables are used in query bodies and heads.

Inference variables $\mathcal{V}_I$: each carries a class annotation $S \in \mathcal{S}$ and constructor keyword arguments $\bar{k}$ mapping field names to body variables. An inference variable carries no pre-specified domain; its domain is populated at rule invocation time:

$$D_{x_I} = \{ H(\bar{k}(\bar{a})) \mid \bar{a} \in \text{ans}(Q_{\text{body}}, \mathcal{K}) \}$$

In EQL, inference variables are constructed via `inference( $H$ )(**kwargs)`. For a plain class H , a derived instance is identified, like a Skolem term $f_H(\bar{k})$, by its class and its argument binding, in which objects are compared by identity and literal values by value and type: evaluating the rule again, or another rule with the same class and binding, returns the same object as long as that object is still referenced. This reuse can be switched off with `inference( $H$ , reuse_inferred_objects=False)`; predicates and symbolic functions are called on every answer.

3 Syntax

This section defines the *abstract syntax* of EQL: the grammar of well-formed terms, atoms, formulas, queries, and rules, independently of their Python surface representation. The Python API constitutes a concrete realisation of this abstract language; where relevant, correspondences to specific API constructs are noted as glosses (e.g., $\wedge$ is expressed in EQL via `and_()` or multiple arguments to `.where()`).

3.1 Terms

Terms are defined inductively over Σ :

$$t ::= c \mid x \mid t.m \mid t.m[i] \mid t.m(\bar{t}) \mid f(t_1, \dots, t_n)$$

where:

- $c \in \mathcal{C}$ is a constant (ground object), also called a *Literal* in EQL; e.g., a specific `Robot` instance.
- $x \in \mathcal{V}$ is a variable with a class and a domain, e.g. r in `variable(Robot, domain=robots)`.
- $t.m$ is a field access on term t , returning the value of field m ; e.g., `r.battery`. Access chains such as $t.m.b$ are admitted, since $t.m$ is itself a term; e.g., `r.tasks[0].completed`.
- $t.m[i]$ is an index access retrieving the element at position i (or with key i for a dictionary) of the indexable field m of t ; e.g., `r.tasks[0]`.
- $t.m(\bar{t})$ is a method call invoking method m on t with arguments $\bar{t}$; when t or any element of $\bar{t}$ is an unground variable, the call is deferred; e.g., `r.name.startswith("R")`.
- $f(t_1, \dots, t_n)$ is an explicit symbolic function application; when any t_i is an unground variable, evaluation is deferred; e.g., `distance(r.position, base)`.

3.2 Atomic Formulas

An *atom* has one of the following forms:

$$\alpha ::= p(t_1, \dots, t_n) \mid t_1 \text{ op } t_2 \mid [t]_{\mathbb{B}}$$

- $p(t_1, \dots, t_n)$ is a *predicate atom*: it denotes the instance of predicate class p constructed with terms $t_1, \dots, t_n$ as arguments. Its truth value is determined at evaluation time by Boolean-casting the ground instance (Section 5.7); e.g., `HasIncompleteTask(r)`.
- $t_1 \text{ op } t_2$ where $\text{op} \in \{=, \neq, <, \leq, >, \geq, \in\}$ is a *comparison atom* over two terms; e.g., `r.battery > 50`.
- $[t]_{\mathbb{B}}$ is a *Boolean term atom*: any term t that evaluates to a Boolean value may appear as an atom by Boolean-casting its value. This covers symbolic function calls ($f(t_1, \dots, t_n)$ where f returns `bool`), method calls ($t.m(\bar{u})$ returning `bool`), and Boolean field accesses ($t.m$ of Boolean type); e.g., `r.name.startswith("R")` or `is_valid(r)`.

Remark 3.1 (Built-in predicates). EQL provides built-in predicate implementations for common TBox queries, including `HasType`, which wraps `isinstance`, and `IsSubclass`, which wraps `issubclass`. These are members of $\mathcal{P}$ like any user-defined predicate and receive no special syntactic treatment.

3.3 Formulas (Query Bodies)

A *body formula* ϕ is built from atoms by:

$$\phi ::= \alpha \mid \phi_1 \wedge \phi_2 \mid \phi_1 \vee \phi_2 \mid \neg \phi \mid \exists x. \phi \mid \forall x. \phi$$

- Conjunctions ($\wedge$) are expressed via multiple arguments to `.where()` or `and_()`.
- Disjunctions ($\vee$) are expressed via `or_()`.
- Negation ($\neg$) is interpreted as Negation as Failure (Section 5.5), expressed via `not_()`.
- Existential quantification is written `exists(x,  $\phi$ )`. It also arises implicitly: a body variable that is not selected in the head is existentially quantified ($\bar{y}$ in Section 3.4).
- Universal quantification is written `for_all(x,  $\phi$ )` and is evaluated over the finite domain of x .
- The domain of a quantified variable may depend on outer variables, for example the courses of a student s , written `flat_variable(s.takes_course)`. Such a quantifier is evaluated separately for each binding of the outer variables. An outer variable stays free in the quantified formula only if it also occurs outside it; a variable that occurs only inside the quantified formula, including in the quantified variable's domain, is quantified together with it.

3.4 Queries

A *query* is:

$$Q(\bar{x}) \leftarrow \phi(\bar{x}, \bar{y})$$

where $\bar{x}$ are the head (output) variables, $\bar{y}$ are body-only existentially quantified variables, and ϕ is a body formula. It is a *conjunctive query* (CQ) if ϕ uses only atoms, conjunction and existential quantification.

A *union of conjunctive queries* (UCQ) is a finite disjunction of CQ bodies sharing the same head variables, expressed in EQL via `or_()` in a single `.where()` call:

$$Q(\bar{x}) \leftarrow \phi_1(\bar{x}) \vee \dots \vee \phi_k(\bar{x})$$

The same-head requirement ensures the result is a well-typed relation: all disjuncts contribute tuples of the same arity and type. In EQL this constraint is enforced architecturally: `or_()` operates inside the body formula ϕ of a single `entity()` or `set_of()` call, so the head variables are shared by construction across all disjuncts.

3.5 Result Quantifiers

A *result quantifier* wraps a query and enforces a cardinality constraint on its answer set, producing a *quantified query* that is itself a query. The constraint is checked incrementally while answers are produced. When it is violated, evaluation raises a `QuantificationNotSatisfiedError`; answers produced before the violation was detected have already been delivered (for example, `.first()` of a `the` query with two answers returns the first).

A *cardinality constraint* $C : \mathbb{N} \rightarrow \{\top, \perp\}$ is one of:

$$C ::= \text{AtLeast}(n) \mid \text{AtMost}(n) \mid \text{Exactly}(n) \mid \text{Range}(\text{AtLeast}(m), \text{AtMost}(n))$$

with the obvious interpretations over the cardinality $|\text{ans}(Q, \mathcal{K})|$.

A *quantified query* pairs a query Q with a constraint C :

$$\mathcal{Q}(Q, C) = \begin{cases} \text{ans}(Q, \mathcal{K}) & \text{if } C(|\text{ans}(Q, \mathcal{K})|) \\ \text{error} & \text{otherwise} \end{cases}$$

EQL provides two quantifier constructors. `an` is the general form: without a constraint argument it applies the trivially satisfied constraint $C(k) = \top$ for all k , acting as a declarative annotation that zero or more results are expected. With a constraint argument it enforces the given C . `the` takes no constraint argument and asserts that exactly one result exists (the constraint `ExactlyOne`); it raises `NoSolutionFound` if there is none and `MultipleSolutionFound` if there are more. It corresponds to Russell’s definite description $\iota x. \phi(x)$ [11], which denotes the unique individual satisfying ϕ and is undefined if no such individual exists or more than one does.

3.6 Rules

A *rule* pairs a query body with an inference variable that constructs new instances from the body’s answer set:

$$x_I \leftarrow \phi(\bar{x}, \bar{y}) \quad x_I \in \mathcal{V}_I, \quad \bar{x} \in \mathcal{V}_Q$$

Rules are not stored in $\mathcal{K}$. On user invocation, the engine evaluates ϕ over the current $\mathcal{A}$, uses the answer set to populate D_{x_I} , and returns the resulting instances, identified like Skolem terms (Section 2.3). The derived instances are ordinary objects (Section 2). There is no fixpoint iteration; each invocation is a single query evaluation.

4 Illustrative Example

The following example is given to ground the abstract definitions in Sections 3 and 5. The domain in this example consists of two classes:

```

@dataclass
class Task:
    name: str
    completed: bool

@dataclass
class Robot:
    name: str
    battery: int
    tasks: List[Task]

```

The fact base $\mathcal{A}$ contains three `Robot` instances and two `Task` instances. The class hierarchy $\mathcal{T}$ contains `Robot` and `Task` as class symbols.

Example query. Retrieve all robots whose battery exceeds 50 and whose first task is not yet completed:

```

r = variable(Robot, domain=robots)
results = an(entity(r).where(r.battery > 50, not_(r.tasks[0].completed))
            ).tolist()

```

In abstract syntax, this is the CQ $Q(r) \leftarrow \phi(r)$ where:

$$\phi(r) = (r.battery > 50) \wedge \neg(r.tasks[0].completed)$$

Here $r \in \mathcal{V}_Q$, $r.battery$ is a field access term (Section 3), $r.tasks[0].completed$ is an index access chained with a field access, $r.battery > 50$ is a comparison atom, and $\neg$ is NAF negation (Section 5.5).

Example rule. Derive `HighLoadRobot` instances for robots whose battery is below 30 and that have at least one incomplete task:

```

@dataclass
class HighLoadRobot:
    robot: Robot

@symbolic_function
def has_incomplete_task(robot):
    return any(not t.completed for t in robot.tasks)

r = variable(Robot, domain=robots)
high_load = inference(HighLoadRobot)(robot=r)
rule = an(entity(high_load).where(r.battery < 30, has_incomplete_task(r)
                                ))

```

In abstract syntax this is the rule $x_I \leftarrow \phi(r)$ where:

$$\phi(r) = (r.battery < 30) \wedge [has_incomplete_task(r)]_{\mathbb{B}}$$

Here $r \in \mathcal{V}_Q$ is a query variable, $x_I \in \mathcal{V}_I$ is an inference variable of class `HighLoadRobot` with argument mapping $robot \mapsto r$ (Section 2.3), and $[has_incomplete_task(r)]_{\mathbb{B}}$ is a Boolean term atom over a symbolic function (Section 3.2). For each binding of r satisfying ϕ , the rule derives the instance `HighLoadRobot(robot=r)`; these instances form the rule's answer set D_{x_I} , and evaluating the rule again returns the same instances.

5 Semantics

5.1 Domain Closure and Unique Names

Definition 5.1 (Domain Closure Assumption). Let $\tau : \mathcal{C} \rightarrow \mathcal{S}$ be a *typing function* that assigns each constant its most specific class. The *domain* of class $S \in \mathcal{S}$ is:

$$\text{dom}(S) = \{ c \in \mathcal{C} \mid \tau(c) \sqsubseteq S \}$$

where $\sqsubseteq$ is the subsumption ordering of $\mathcal{T}$. No individuals exist beyond those in $\mathcal{A}$.

In EQL, τ is realised by Python’s class system and domain membership $\tau(c) \sqsubseteq S$ can be checked at runtime using `isinstance(c, S)`.

Every query variable $x \in \mathcal{V}_Q$ ranges over a finite domain, given or taken from the objects in memory. Inference variables $x_I \in \mathcal{V}_I$ are domain-free at construction; their domain is populated from the rule body’s answer set at invocation time.

Objects and individuals. Objects and individuals do not correspond one to one in KR-ROOD, because of the Role pattern: a role object refers to the object playing it, its *role taker*. We write $\text{ind}(o)$ for the individual that object o denotes: a role object denotes the individual of its role taker, an object created for an ontology individual denotes the individual named by its IRI, and every other object denotes an individual of its own; on data values, ind is the identity. The *unique name assumption* reads: two objects denote the same individual only if they carry the same IRI or one is a role of the other, directly or through further roles (a role can itself have a role). Together with domain closure, this fixes a single finite interpretation $\mathcal{I}_{\mathcal{A}}$ of $\mathcal{K}$ [2]: a class denotes the individuals of its instances, an attribute relates the individual of every object to the individuals of its values, a predicate denotes the tuples for which its code returns true, and a symbolic function the function its code computes. The extensions of predicates and functions are not stored but computed when evaluation needs them.

5.2 Evaluation Semantics

EQL evaluates queries by nested iteration over the variables’ domains, a left-deep nested-loop join. Domains may be any iterable; their finiteness is a precondition. With finite domains, evaluation makes finitely many invocations of user code (Section 8), so it terminates whenever every invoked piece of user code terminates (Section 2).

Given $Q(\bar{x}) \leftarrow \phi(\bar{x}, \bar{y})$ and $\mathcal{K} = (\mathcal{T}, \mathcal{A})$, evaluation proceeds left to right through the expression tree of ϕ :

1. **Lazy variable binding.** Variables are not enumerated upfront. Each variable $x_i \in \mathcal{V}_Q$ is bound when its domain D_i is first encountered in the left-to-right traversal of ϕ . The engine iterates over D_i in a nested loop, passing each candidate binding forward to the next subexpression.
2. **Short-circuit filtering.** Under $\wedge$: if a subexpression fails for the current binding θ , the remaining conjuncts are not evaluated for that θ , and the next candidate is tried. Under $\vee$: if the left disjunct succeeds, the right is skipped. No enumeration is performed beyond what the expression requires.
3. **Head projection.** After ϕ is satisfied, any head variable $x_i \in \bar{x}$ not constrained by ϕ is joined with the current bindings via a Cartesian product over D_i to produce complete result tuples.
4. **Result collection.** All bindings θ for which ϕ evaluates to true are collected; projecting onto $\bar{x}$ yields the answer set.

5.3 Answer Set

Definition 5.2 (Answer Set). The *answer set* of query $Q(\bar{x}) \leftarrow \phi(\bar{x}, \bar{y})$ with respect to $\mathcal{K}$ is:

$$\text{ans}(Q, \mathcal{K}) = \{ \theta(\bar{x}) \mid \mathcal{I}_{\mathcal{A}} \models \phi\theta \}$$

where θ ranges over the assignments of values to all variables $\bar{x}\bar{y}$ of the head and the body with $\theta(x) \in \llbracket D_x \rrbracket_{\theta}$ for each of them (a domain may depend on other variables, also on body variables, as in `flat_variable(dean.teaches_course)`), $\models$ is first-order satisfaction, and the quantifiers in ϕ range over the variables' domains in the same way. Negation is negation as failure (Section 5.5).

As in SQL, evaluation produces answers with bag semantics, one for every satisfying binding of the body's variables; `distinct` removes duplicates and yields the set above.

5.4 Aggregation Semantics

An *aggregated query* is a semantic transformation applied to the answers of a query, corresponding to the γ operator of relational algebra [5, 2] and to SQL's `GROUP BY` pattern.

Let $\text{Agg} = \{\text{count}, \text{sum}, \text{avg}, \text{max}, \text{min}, \text{mode}, \text{multimode}\}$ be the set of aggregate functions, each mapping a multiset of values to a scalar or set of scalars. Given a query $Q(\bar{x}) \leftarrow \phi(\bar{x}, \bar{y})$, a grouping key $\bar{g} \subseteq \bar{x}$, and aggregate expressions $f_1(e_1), \dots, f_k(e_k)$ where each e_i is a term over $\bar{x}\bar{y}$ (e.g. `count(c)` for a body variable c), the aggregated query evaluates as:

$$\gamma_{\bar{g}; e_1 \rightarrow f_1, \dots, e_k \rightarrow f_k}(B(Q, \mathcal{K}))$$

where $B(Q, \mathcal{K})$ is the bag of satisfying bindings that evaluation produces. This partitions $B(Q, \mathcal{K})$ by $\bar{g}$ and computes each f_i over the multiset of e_i values within each group. In EQL this is expressed via `set_of(...).grouped_by( $\bar{g}$ )`. Post-aggregation filtering via `.having()` applies a condition after aggregation, equivalent to SQL's `HAVING`; this is distinct from `.where()`, which filters tuples before grouping as part of ϕ . When no grouping key is specified, f is computed over the entire bag $B(Q, \mathcal{K})$.

5.5 Negation as Failure

EQL supports *Negation as Failure* (NAF) [4]. In Definition 5.2, $\neg\phi\theta$ holds if and only if $\phi\theta$ is not satisfied in $\mathcal{I}_{\mathcal{A}}$. Because $\mathcal{I}_{\mathcal{A}}$ is a single interpretation in which every ground atom, computed atoms included, is either true or false, this is Reiter's *Closed World Assumption* (CWA) [6]: for a database of ground atoms under domain closure and unique names, the CWA determines a model that is unique up to isomorphism. Operationally, $\neg\phi$ succeeds in a query body if and only if ϕ fails finitely during evaluation. NAF is expressed in EQL via `not_()`.

NAF is **non-monotonic**: adding new facts to $\mathcal{A}$ may invalidate previously derived negative conclusions. EQL does not provide monotonic or paraconsistent negation.

5.6 Semantics of Terms

Under a ground substitution θ , terms evaluate as follows.

Field access.

$$\llbracket t.m \rrbracket_{\theta} = \text{the value of field } m \text{ on object } \llbracket t \rrbracket_{\theta}$$

Access chains evaluate left-associatively: $\llbracket t.m.b \rrbracket_{\theta} = \llbracket (t.m).b \rrbracket_{\theta}$.

Index access.

$$\llbracket t.m[i] \rrbracket_{\theta} = \llbracket t.m \rrbracket_{\theta}[\llbracket i \rrbracket_{\theta}]$$

i.e., element i of the indexable value of field m on $\llbracket t \rrbracket_{\theta}$.

Method call.

$$\llbracket t.m(\bar{u}) \rrbracket_\theta = \llbracket t.m \rrbracket_\theta(\llbracket \bar{u} \rrbracket_\theta)$$

i.e., method m of $\llbracket t \rrbracket_\theta$ called with ground arguments $\llbracket \bar{u} \rrbracket_\theta$. If t or any element of $\bar{u}$ is unground, the call is deferred as a symbolic function application, returning a new EQL variable.

Symbolic function.

$$\llbracket f(t_1, \dots, t_n) \rrbracket_\theta = f_{\text{impl}}(\llbracket t_1 \rrbracket_\theta, \dots, \llbracket t_n \rrbracket_\theta)$$

where f_{impl} is the underlying callable. If any t_i is unground, evaluation is deferred. Determinism and purity ensure the result is unique and $\mathcal{A}$ is unmodified.

5.7 Semantics of Atoms

Atom evaluation determines whether a ground atom holds under substitution θ . Term evaluation (Section 5.6) is applied first to obtain ground values; truth determination is a separate subsequent step.

Predicate atom. Applying θ to $p(t_1, \dots, t_n)$ yields the ground predicate instance:

$$\llbracket p(t_1, \dots, t_n) \rrbracket_\theta = p(\llbracket t_1 \rrbracket_\theta, \dots, \llbracket t_n \rrbracket_\theta)$$

The atom *holds* if and only if the Boolean cast of this instance returns **True**:

$$\alpha \text{ holds under } \theta \iff \text{bool}(\llbracket p(t_1, \dots, t_n) \rrbracket_\theta) = \text{True}$$

The Boolean cast invokes the characteristic function `__call__` of the instance.

Comparison atom. The atom $t_1 \text{ op } t_2$ holds under θ if and only if:

$$\llbracket t_1 \rrbracket_\theta \text{ op } \llbracket t_2 \rrbracket_\theta$$

Boolean term atom. The atom $[t]_{\mathbb{B}}$ holds under θ if and only if:

$$\text{bool}(\llbracket t \rrbracket_\theta) = \text{True}$$

This applies to any term evaluating to a Boolean value: symbolic function calls, method calls, and Boolean field accesses are all admitted as atoms via this form.

6 Reasoning on Assignment

This section formalizes what the property descriptors of KRROOD compute when application code assigns a value, and proves the runtime proposition of the paper (Proposition 5.3). It follows the implementation of the version used in the experiments (`PropertyDescriptorRelation.infer_and_apply` in `krrood/ontomatic/property_descriptor/property_descriptor_relation.py`).

6.1 Property Facts and Rules

Let $\mathbf{P}$ be the object properties of the TBox. Each $P \in \mathbf{P}$ has super-properties $\text{sup}(P)$ (the transitive closure of the descriptor hierarchy), equivalent properties $\text{eq}(P)$, at most one inverse $\text{inv}(P)$, and may be *symmetric* or *transitive*; a *chain* $Q_1 \circ \dots \circ Q_n \sqsubseteq P$ has $n \geq 2$. A *fact* is a triple $P(a, b)$ of a property and two individuals; the facts of a knowledge base are the pairs of Section 3 of the paper, $P(\text{ind}(o), \text{ind}(v))$ for every object o and value v of the attribute that P manages.

The *assign rules* are the following Horn rules, which are the OWL 2 RL/RDF rules **prp-spo1**, **prp-eqp1/2**, **prp-inv1/2**, **prp-symp**, **prp-trp** and **prp-spo2** instantiated for the declared characteristics:

$$\begin{array}{ll}
P(a, b) \rightarrow S(a, b) & S \in \text{sup}(P) \\
P(a, b) \rightarrow E(a, b) & E \in \text{eq}(P) \\
P(a, b) \rightarrow \text{inv}(P)(b, a) & \\
P(a, b) \rightarrow P(b, a) & P \text{ symmetric} \\
P(a, b) \wedge P(b, c) \rightarrow P(a, c) & P \text{ transitive, not symmetric} \\
Q_1(a_0, a_1) \wedge \dots \wedge Q_n(a_{n-1}, a_n) \rightarrow P(a_0, a_n) & Q_1 \circ \dots \circ Q_n \sqsubseteq P
\end{array}$$

We write R for this set of rules. Transitivity of a property that is also symmetric is not in R : it is completed by step 5 of the loading algorithm, which relates all members of each connected component, and not on assignment.

Storage condition. A fact $P(a, b)$ can be stored only on an object of a whose class declares the attribute that P manages. We write $\text{decl}(a, P)$ if such an object exists. Classes of objects do not change, so decl does not change while facts are assigned.

6.2 The Assignment Procedure

Definition 6.1 (Assignment). Let F be a set of facts and f a fact asserted by application code, with decl true for its subject and property. $\text{Assign}(F, f)$ adds f to F and then *propagates* it. Propagating a fact g means: for every rule $r \in R$ and every instance of r in which g is one of the premises and every other premise is in the current F , let h be the conclusion; if decl holds for the subject and property of h and $h \notin F$, add h to F and propagate h before continuing.

The definition is the depth-first recursion of the implementation: a derived fact is stored (**update_source**) and its implications are applied at once (**infer_and_apply_implications**); a fact already present is not propagated again; and a fact whose subject has no object declaring the property is not stored (**holder_of_property** returns **None**). The implementation skips some rule instances whose conclusion is certainly present, such as the symmetric fact of a fact derived by symmetry; these skips do not change the result. The procedure terminates because F grows in every step and there are finitely many facts over the finitely many individuals and properties.

Remark 6.2. The implementation does not apply transitivity to a fact that it derived from an equivalent property. Proposition 6.4 therefore assumes that equivalent properties have the same characteristics, so that the fact from which the equivalent one was derived is chained instead.

6.3 Soundness and Closure at Runtime

Proposition 6.3 (Runtime soundness). *Let $\mathcal{O}$ be an OWL 2 RL ontology, F_0 the facts of the knowledge base that Ontomatic loads from it, and $f_1, \dots, f_m$ facts asserted by application code. Every fact of $F_m = \text{Assign}(\dots \text{Assign}(F_0, f_1) \dots, f_m)$ is entailed by $\mathcal{O}$ together with $f_1, \dots, f_m$.*

Proof. By Proposition 5.1 of the paper, every fact of F_0 is entailed by $\mathcal{O}$. Every fact that an assignment adds is either an asserted f_i or the conclusion of an instance of a rule of R whose premises were in F when it was added. Each rule of R is an OWL 2 RL/RDF rule, and these rules are sound for the direct semantics [14]. By induction on the order in which facts are added, every fact is entailed. $\square$

Proposition 6.4 (Runtime closure). *Let F be closed under R , let equivalent properties have the same characteristics, and let f be an asserted fact. If decl holds for the subject and property of every conclusion of an instance of a rule of R whose premises are in $\text{Assign}(F, f)$, then $\text{Assign}(F, f)$ is closed under R .*

Proof. Let $F' = \text{Assign}(F, f)$ and suppose that an instance of a rule of R has premises $p_1, \dots, p_k \in F'$ and a conclusion $h \notin F'$. If all premises are in F , then $h \in F$ because F is closed, a contradiction. Otherwise, let p_j be the premise that the procedure added last. Every other premise was in F or was added before p_j , so it was present when p_j was propagated, which happens right after p_j is added. Propagating p_j considers every instance of every rule in which p_j is a premise and the others are present, including this one; *decl* holds for h by assumption, so h was added, a contradiction. For a transitive property that is equivalent to another, a premise derived through the equivalence is not chained, but by the remark its source fact has the same characteristic and is chained, and the conclusion is carried over by the equivalence rule. $\square$

By induction on the asserted facts, the closure under R that loading establishes (steps 1–4 of the loading algorithm, Proposition 5.2 of the paper) is kept after any sequence of assignments that satisfies the storage condition. The guarantee concerns object-property facts and the rules of R . It does not cover:

- transitivity of properties that are both symmetric and transitive, which step 5 completes only when loading;
- class memberships that the OWL 2 RL/RDF rules derive from the new facts (**prp-dom**, **prp-rng**, **cls-***, **cax-***): the class of an object does not change after it is created. A class defined by an axiom is instead queried through its axiom, which evaluates the same condition on the current facts;
- facts whose subject has no object declaring the property, which are not stored;
- sub- and equivalent data properties, whose values Ontomatic copies when loading.

The runtime audit of the supplementary material measures how many facts each of these leaves out after the agent loop of the paper.

7 The OWL 2 RL/RDF Rules in Ontomatic

The OWL 2 RL/RDF rules [14] are 78 first-order rules over RDF triples, grouped in six tables of Section 4.3 of the specification: equality (Table 4, **eq-***), axioms about properties (Table 5, **prp-***), classes (Table 6, **cls-***), class axioms (Table 7, **cax-***), datatypes (Table 8, **dt-***) and the schema vocabulary (Table 9, **scm-***). The tables below list every rule and how the version of KRROOD used in the experiments (Ontomatic) realizes it. In the column “derives”, $C(x)$ and $P(x, y)$ are class and property assertions, $x \approx y$ is **owl:sameAs**, $x \not\approx y$ is **owl:differentFrom**, and $\perp$ is the contradiction **false**. “Step” refers to the loading procedure (Algorithm 1 of the paper, **OwlLoader.load**). Each rule has one of the following mechanisms.

LOAD Applied by the loading procedure: on the proxies of the individuals in steps 1–2 (types, and the values that **hasValue** restrictions and chains add), and for the facts of **hasValue** restrictions also on the objects in step 4. Not applied after loading: the class of an object does not change.

ASSIGN Applied by the property descriptors whenever a fact is assigned, at load time (step 4; also on the proxies in steps 1–2) and at runtime. These are the rules R of Section 6.

SCHEMA Holds by construction of the generated model: class and descriptor inheritance, aliases for equivalent classes, and **issubclass_or_role**, which follows inheritance and role-taker chains. No step materializes the conclusion.

CHECK Not applied. The check after loading (**OwlInstancesRegistry.check_owl2_rl**, module **owl2rl_check**) evaluates the rule on the loaded knowledge base and reports every application: an equality of two individuals or an inconsistency.

NONE Derives no class or property assertion about individuals that is not already present, provided the check reports no equality: the conclusion is a reflexive `owl:sameAs`, an axiomatic triple, or follows from an equality of two different individuals.

PYTHON Concerns literals, which are Python values: datatype membership is the Python type of the value, and equality of values is Python equality.

EXCLUDED Outside the conditions of Theorem PR1 of the specification, under which the paper's propositions are stated (no annotation-property axioms, no punning).

UNSUPPORTED Not realized; the note says what happens.

A rule marked [†] is realized only for the forms of its construct given in the note. The mechanisms follow the code of the earlier version in the supplement: `owl_to_python.py` (TBox compilation), `owl_instances_loader.py` and `owl2rl_tbox.py` (loading), `property_descriptor_relation.py` (assignment) and `owl2rl_check.py` (check).

Mechanism	eq	prp	cls	cax	dt	scm	Total
LOAD	0	2	6	0	0	6	14
ASSIGN	0	8	0	0	0	1	9
SCHEMA	0	0	1	3	0	11	15
CHECK	3	9	7	2	0	0	21
NONE	6	0	2	0	1	0	9
PYTHON	0	0	0	0	3	0	3
EXCLUDED	0	1	0	0	0	0	1
UNSUPPORTED	0	0	3	0	1	2	6
Total	9	20	19	5	5	20	78
of which [†]	0	7	8	2	0	1	18

Table 1: The semantics of equality (OWL 2 Profiles, Table 4).

Rule	Derives	KRROOD	Note
eq-ref	$x \approx x$ for every term	NONE	Reflexive <code>owl:sameAs</code> is object identity and is not stored; the closure argument of Proposition 5.2 adds these facts implicitly.
eq-sym	$x \approx y \Rightarrow y \approx x$	NONE	KRROOD never merges individuals (unique names). The check reports every <code>owl:sameAs</code> between different IRIs that is asserted or derived by <code>prp-fp</code> , <code>prp-ifp</code> , <code>prp-key</code> , <code>cls-maxc2</code> or <code>cls-maxqc3/4</code> . If it reports none, only reflexive equalities hold, from which this rule and the next four derive nothing new.
eq-trans	$x \approx y, y \approx z \Rightarrow x \approx z$	NONE	As eq-sym .
eq-rep-s	$x \approx x', P(x, y) \Rightarrow P(x', y)$	NONE	As eq-sym .
eq-rep-p	$P \approx P', P(x, y) \Rightarrow P'(x, y)$	NONE	As eq-sym ; an equality of two properties would moreover need punning.
eq-rep-o	$y \approx y', P(x, y) \Rightarrow P(x, y')$	NONE	As eq-sym .

Rule	Derives	KRROOD	Note
eq-diff1	$x \approx y, x \not\approx y \Rightarrow \perp$	CHECK	Each pair of <code>owl:differentFrom</code> individuals is compared with the equalities that the check found. Equalities are not closed under eq-trans , but any equality already fails the check.
eq-diff2	equal members of <code>owl:AllDifferent</code> (<code>owl:members</code>) $\Rightarrow \perp$	CHECK	As eq-diff1 .
eq-diff3	the same with <code>owl:distinctMembers</code>	CHECK	As eq-diff1 .

Table 2: The semantics of axioms about properties (OWL 2 Profiles, Table 5).

Rule	Derives	KRROOD	Note
prp-ap	built-in annotation properties are annotation properties	EXCLUDED	Axiomatic triples about annotation properties. Ontomatic compiles no annotation properties; <code>rdfs:label</code> and <code>rdfs:comment</code> become docstrings.
prp-dom[†]	$P(x, y) \Rightarrow C(x)$ for domain C of P	LOAD (step 2)	Object and data properties. Facts of super-, equivalent and inverse properties and of symmetry are added in step 1, so their domains apply as well. Only named domain classes are used; a class expression as domain is ignored. Not applied at runtime.
prp-rng[†]	$P(x, y) \Rightarrow C(y)$ for range C of P	LOAD (step 2)	Object properties, named range classes, as prp-dom . The range of a data property is a datatype (dt-type2 , dt-not-type).
prp-fp	functional P : $P(x, y_1), P(x, y_2) \Rightarrow y_1 \approx y_2$	CHECK	Object property: an equality is reported. Data property: two different values are an inconsistency (with dt-diff and eq-diff1).
prp-ifp	inverse-functional P : $P(x_1, y), P(x_2, y) \Rightarrow x_1 \approx x_2$	CHECK	Equality reported.
prp-irp[†]	irreflexive P : $P(x, x) \Rightarrow \perp$	CHECK	Missed for a property that is symmetric, transitive and irreflexive: step 5 omits the facts $P(x, x)$ of such a property, so a non-empty one, which is inconsistent, passes the check.
prp-symp	symmetric P : $P(x, y) \Rightarrow P(y, x)$	ASSIGN (step 1)	Mixin SymmetricProperty .
prp-asymp	asymmetric P : $P(x, y), P(y, x) \Rightarrow \perp$	CHECK	Also reports $P(x, x)$.
prp-trp	transitive P : $P(x, y), P(y, z) \Rightarrow P(x, z)$	ASSIGN; step 2; step 5	Mixin TransitiveProperty . On assignment for properties that are not symmetric; in step 2 on the proxies for the properties used by class expressions and chains; in step 5 (weakly connected components) for symmetric ones, at load time only.

Rule	Derives	KRROOD	Note
prp-spo1	$P_1 \sqsubseteq P_2$: $P_1(x, y) \Rightarrow P_2(x, y)$	ASSIGN (step 1)	Descriptor inheritance. Data properties: the value is copied to the attributes of the implied properties when loading (step 4), not at runtime.
prp-spo2	chain $P_1 \circ \dots \circ P_n \sqsubseteq P$	ASSIGN (step 2)	Mixin HasChainAxioms ; in step 2 also on the proxies.
prp-eqp1	$P_1 \equiv P_2$: $P_1(x, y) \Rightarrow P_2(x, y)$	ASSIGN (step 1)	Mixin HasEquivalentProperties . Data properties are copied when loading, as for prp-spo1 .
prp-eqp2	$P_1 \equiv P_2$: $P_2(x, y) \Rightarrow P_1(x, y)$	ASSIGN (step 1)	As prp-eqp1 .
prp-pdw[†]	disjoint P_1, P_2 : $P_1(x, y), P_2(x, y) \Rightarrow \perp$	CHECK	Object properties only; values of disjoint data properties are not compared.
prp-adp[†]	the same for owl:AllDisjointProperties	CHECK	As prp-pdw .
prp-inv1[†]	P_1 inverse of P_2 : $P_1(x, y) \Rightarrow P_2(y, x)$	ASSIGN (step 1)	Mixin HasInverseProperty , with at most one inverse per property: if P has two inverses Q_1, Q_2 , only one of $Q_1(y, x), Q_2(y, x)$ is derived from $P(x, y)$. Only named inverses (no ObjectInverseOf expressions).
prp-inv2[†]	P_1 inverse of P_2 : $P_2(x, y) \Rightarrow P_1(y, x)$	ASSIGN (step 1)	As prp-inv1 .
prp-key	hasKey : same key values $\Rightarrow x \approx y$	CHECK	Object and data key properties.
prp-npa1	negative object property assertion violated $\Rightarrow \perp$	CHECK	
prp-npa2	negative data property assertion violated $\Rightarrow \perp$	CHECK	Values compared as Python values (dt-eq).

Table 3: The semantics of classes (OWL 2 Profiles, Table 6).

Rule	Derives	KRROOD	Note
cls-thing	owl:Thing is a class	NONE	Axiomatic. owl:Thing corresponds to the generated base class of the ontology, which every generated class inherits; an individual without any type is an object of the base class (end of step 2).
cls-nothing1	owl:Nothing is a class	NONE	Axiomatic.
cls-nothing2	$Nothing(x) \Rightarrow \perp$	UNSUPPORTED	owl:Nothing has no generated class: a class declared a subclass of it gets an undefined base class and the generated model fails to import, and an assertion rdf:type owl:Nothing is ignored without report.

Rule	Derives	KRROOD	Note
cls-int1 [†]	$C \equiv C_1 \sqcap \dots \sqcap C_n:$ $C_1(y), \dots, C_n(y) \Rightarrow C(y)$	LOAD (axiom, step 2)	An intersection in subclass position that contains a restriction becomes the class's axiom; its named conjuncts become type checks (Student in Student $\sqcap \exists \text{hasMajor.Science} \sqsubseteq$ ScienceStudent). An intersection of named classes only ($A \sqcap B \sqsubseteq C$) yields no axiom and derives nothing. A class owl:equivalentClass to an intersection makes TBox compilation fail. The restrictions on the class (see cls-svf1) apply.
cls-int2 [†]	$C(y) \Rightarrow C_1(y), \dots, C_n(y)$	LOAD (step 2)	Intersections in superclass position (necessary conditions), applied on the proxies and their absence reported by the check. Not for owl:equivalentClass to an intersection (see cls-int1).
cls-uni [†]	$C \equiv C_1 \sqcup \dots \sqcup C_n:$ $C_i(y) \Rightarrow C(y)$	SCHEMA	A union in subclass position with named superclasses: every operand inherits these superclasses, so cls-uni and cax-sco hold by inheritance. A union inside another class expression is not compiled.
cls-com [†]	$C_1 \equiv \neg C_2:$ $C_1(x), C_2(x) \Rightarrow \perp$	CHECK	ObjectComplementOf of a named class in superclass position, also as the filler of an allValuesFrom restriction (womanCollege in OWL2Bench). The complement of a class expression is not checked.
cls-svf1 [†]	$\exists P.D \sqsubseteq C:$ $P(u, v), D(v) \Rightarrow C(u)$	LOAD (axiom, step 2)	someValuesFrom of an object property with a named filler, in subclass position. The axiom tests the filler with issubclass_or_role . The generated axiom of a class is the conjunction of all its sufficient conditions, so for a class with two (e.g. $\exists P.D \sqsubseteq C$ and $\exists Q.D \sqsubseteq C$) an individual satisfying one of them is not classified. Loading fails if the property has a declared domain that is not a superclass of the class, which then declares no attribute for it (observed for hasValue , which takes the same code path).
cls-svf2	$\exists P.\top \sqsubseteq C:$ $P(u, v) \Rightarrow C(u)$	UNSUPPORTED	The filler owl:Thing is compiled to the name Thing , which is not a generated class; loading fails.
cls-avf	$C \sqsubseteq \forall P.D:$ $C(u), P(u, v) \Rightarrow D(v)$	LOAD (step 2)	allValuesFrom in superclass position, applied on the proxies; a hasValue in the filler adds its facts in step 4.
cls-hv1	$C \sqsubseteq \exists P.\{a\}:$ $C(u) \Rightarrow P(u, a)$	LOAD (steps 2, 4)	hasValue in superclass position or in an owl:equivalentClass definition: added to the proxies in step 2 and assigned through the descriptor in step 4, so that the assign rules fire. Object and data properties.

Rule	Derives	KRROOD	Note
cls-hv2[†]	$\exists P.\{a\} \sqsubseteq C:$ $P(u, a) \Rightarrow C(u)$	LOAD (axiom, step 2)	owl:equivalentClass definitions (Listing 3 of the paper) and general class axioms; the restrictions of cls-svf1 on the property apply. At runtime the class is queried through its axiom.
cls-maxc1[†]	$C \sqsubseteq \leq 0 P:$ $C(u), P(u, y) \Rightarrow \perp$	CHECK	Object properties only; maximum cardinalities of data properties are not checked.
cls-maxc2[†]	$C \sqsubseteq \leq 1 P:$ $P(u, y_1), P(u, y_2) \Rightarrow y_1 \approx y_2$	CHECK	Equality reported. Object properties only; for a data property the rule with dt-diff gives an inconsistency, which is not reported.
cls-maxqc1	$C \sqsubseteq \leq 0 P.D \Rightarrow \perp$	CHECK	Equality reported; the values are tested for D on the loaded objects.
cls-maxqc2	$C \sqsubseteq \leq 0 P.\top \Rightarrow \perp$	CHECK	
cls-maxqc3	$C \sqsubseteq \leq 1 P.D \Rightarrow y_1 \approx y_2$	CHECK	
cls-maxqc4	$C \sqsubseteq \leq 1 P.\top \Rightarrow y_1 \approx y_2$	CHECK	Equality reported.
cls-oo	$\{a_1, \dots, a_n\} \sqsubseteq C: C(a_i)$	UNSUPPORTED	oneOf is not compiled; the individuals are not typed, without report.

Table 4: The semantics of class axioms (OWL 2 Profiles, Table 7).

Rule	Derives	KRROOD	Note
cax-sco	$C_1 \sqsubseteq C_2: C_1(x) \Rightarrow C_2(x)$	SCHEMA	Between named classes: Python inheritance or the Role pattern; an object of C_1 is an instance of C_2 under issubclass_or_role , and only the most specific types become objects (step 3). A subclass of an intersection: cls-int2 (LOAD).
cax-eqc1	$C_1 \equiv C_2: C_1(x) \Rightarrow C_2(x)$	SCHEMA	Equivalent named classes are one Python class; the other name is an alias (School = College in OWL2Bench).
cax-eqc2	$C_1 \equiv C_2: C_2(x) \Rightarrow C_1(x)$	SCHEMA	As cax-eqc1 .
cax-dw[†]	disjoint $C_1, C_2:$ $C_1(x), C_2(x) \Rightarrow \perp$	CHECK	Named classes only; disjointness with a class expression is not checked.
cax-adc[†]	the same for owl:AllDisjointClasses	CHECK	As cax-dw .

Table 5: The semantics of datatypes (OWL 2 Profiles, Table 8).

Rule	Derives	KRROOD	Note
dt-type1	every supported datatype is a datatype	NONE	Axiomatic.
dt-type2	literal ℓ in the value space of $d: d(\ell)$	PYTHON	A literal is stored as a Python value (rdflib 's toPython , coerced to the type annotation of the attribute); its datatype membership is its Python type and is not stored as a fact. No class or property assertion about an individual depends on it.

Rule	Derives	KRROOD	Note
dt-eq	same data value: $\ell_1 \approx \ell_2$	PYTHON	Python equality of the values, used by the check for functional data properties, keys and negative data property assertions.
dt-diff	different data values: $\ell_1 \not\approx \ell_2$	PYTHON	As dt-eq ; two different values of a functional data property are reported as an inconsistency.
dt-not-type	$d(\ell)$, ℓ not in the value space of $d \Rightarrow \perp$	UNSUPPORTED	Not checked: a value outside the declared datatype range of a data property (for example a string for xsd:integer , an inconsistency with prp-rng) is stored after a failed coercion, without report.

Table 6: The semantics of schema vocabulary (OWL 2 Profiles, Table 9). These rules derive schema triples only; the table says how the assertions that follow from them are obtained.

Rule	Derives	KRROOD	Note
scm-cls	$C \sqsubseteq C$, $C \equiv C$, $C \sqsubseteq \top$, $\perp \sqsubseteq C$	SCHEMA	issubclass is reflexive and every generated class inherits the base class (owl:Thing); owl:Nothing is not represented (cls-nothing2).
scm-sco	$C_1 \sqsubseteq C_2 \sqsubseteq C_3 \Rightarrow C_1 \sqsubseteq C_3$	SCHEMA	issubclass is transitive.
scm-eqc1	$C_1 \equiv C_2 \Rightarrow C_1 \sqsubseteq C_2$, $C_2 \sqsubseteq C_1$	SCHEMA	One class with an alias (cax-eqc1).
scm-eqc2	$C_1 \sqsubseteq C_2 \sqsubseteq C_1 \Rightarrow C_1 \equiv C_2$	UNSUPPORTED	Python inheritance cannot be cyclic: for a cycle of rdfs:subClassOf the generated model fails to import. The equivalence must be stated with owl:equivalentClass .
scm-op	$P \sqsubseteq P$, $P \equiv P$ (object property)	SCHEMA	issubclass is reflexive; the conclusions derive no new fact.
scm-dp	the same for data properties	SCHEMA	As scm-op .
scm-spo	$P_1 \sqsubseteq P_2 \sqsubseteq P_3 \Rightarrow P_1 \sqsubseteq P_3$	SCHEMA	Descriptor inheritance: a fact of P_1 propagates to each direct super-property, which propagates further (prp-spo1).
scm-eqp1	$P_1 \equiv P_2 \Rightarrow P_1 \sqsubseteq P_2$, $P_2 \sqsubseteq P_1$	ASSIGN	Equivalent properties are distinct descriptor classes related by HasEquivalentProperties , not by inheritance; their facts are copied by prp-eqp1/2 .
scm-eqp2	$P_1 \sqsubseteq P_2 \sqsubseteq P_1 \Rightarrow P_1 \equiv P_2$	UNSUPPORTED	As scm-eqc2 : a cycle of rdfs:subPropertyOf makes the generated descriptor module fail to import.
scm-dom1	$dom(P) = C_1$, $C_1 \sqsubseteq C_2 \Rightarrow dom(P) = C_2$	SCHEMA	An object typed C_1 by prp-dom is an instance of C_2 .
scm-dom2	$dom(P_2) = C$, $P_1 \sqsubseteq P_2 \Rightarrow dom(P_1) = C$	LOAD (steps 1–2)	Step 1 adds the facts of the super-property, and step 2 applies its domain. TBox compilation also derives undeclared domains of descriptors from super-, inverse and equivalent properties.

Rule	Derives	KRROOD	Note
scm-rng1	$rng(P) = C_1,$ $C_1 \sqsubseteq C_2 \Rightarrow rng(P) = C_2$	SCHEMA	As scm-dom1 .
scm-rng2	$rng(P_2) = C,$ $P_1 \sqsubseteq P_2 \Rightarrow rng(P_1) = C$	LOAD (steps 1–2)	As scm-dom2 .
scm-hv	$P_1 \sqsubseteq P_2 \Rightarrow \exists P_1.\{a\} \sqsubseteq$ $\exists P_2.\{a\}$	LOAD (steps 1–2)	The fact of P_2 is added in step 1 (prp-spo1), so the axiom on P_2 holds (cls-hv2).
scm-svf1	$D_1 \sqsubseteq D_2 \Rightarrow \exists P.D_1 \sqsubseteq$ $\exists P.D_2$	SCHEMA	The axiom tests the filler with issubclass_or_role , so a value of type D_1 satisfies $\exists P.D_2$.
scm-svf2	$P_1 \sqsubseteq P_2 \Rightarrow \exists P_1.D \sqsubseteq$ $\exists P_2.D$	LOAD (steps 1–2)	As scm-hv .
scm-avf1	$D_1 \sqsubseteq D_2 \Rightarrow \forall P.D_1 \sqsubseteq$ $\forall P.D_2$	SCHEMA	A value typed D_1 by cls-avf is an instance of D_2 .
scm-avf2	$P_1 \sqsubseteq P_2 \Rightarrow \forall P_2.D \sqsubseteq$ $\forall P_1.D$	LOAD (steps 1–2)	The fact of P_2 is added in step 1, and cls-avf on P_2 applies in step 2.
scm-int[†]	$C \equiv C_1 \sqcap \dots \sqcap C_n \Rightarrow$ $C \sqsubseteq C_i$	LOAD (step 2)	Obtained as cls-int2 for intersections in superclass position; an owl:equivalentClass to an intersection makes TBox compilation fail.
scm-uni	$C \equiv C_1 \sqcup \dots \sqcup C_n \Rightarrow$ $C_i \sqsubseteq C$	SCHEMA	The operands inherit the superclasses of the union (cls-uni).

8 Computational Complexity

Data complexity fixes the query and measures cost as $|\mathcal{A}|$ grows—the relevant measure for a deployed system. *Combined complexity* lets both the query and $\mathcal{A}$ vary [13]. Table 7 summarises the results; each row adds one feature to the row above, so the bounds can only grow from one row to the next.

Table 7: Complexity of EQL query evaluation by feature class.

Feature class	Data complexity	Combined complexity
CQ and UCQ	AC^0	NP-complete
+ NAF and $\forall$ (first-order logic, active domain)	AC^0	PSPACE-complete
+ aggregation	TC^0	PSPACE-complete ^a
+ predicates and symbolic functions (as oracles)	FP^O	PSPACE-complete ^b

^a For **count**, **sum**, **min**, **max** and **avg** over numbers of polynomial size.

^b If every callable runs in polynomial time and every value computed has size polynomial in the size of $\mathcal{A}$ and the query.

The entries follow from established results:

- **CQ and UCQ.** CQ evaluation is NP-complete in combined complexity [3]; UCQ shares the same bounds, and both are in AC^0 in data complexity [2].
- **+ NAF and $\forall$.** With negation and universal quantification over finite domains, query bodies are first-order formulas evaluated under the active-domain semantics. Their combined

complexity is PSPACE-complete [13]: nested alternations of $\forall$ and $\exists$ encode quantified Boolean formulas, so unfolding $\forall x \in D$ into a finite conjunction does not lower the bound. Their data complexity is in AC^0 [7, 8].

- + **aggregation.** Counting aggregates can express majority, which is not in AC^0 ; first-order logic with aggregates has data complexity in TC^0 [9, 12].
- + **predicates and symbolic functions.** We treat callables as oracles. A query ϕ with k variables, including quantified ones, whose domains have size at most d , invokes callables at most $|\phi| \cdot d^k$ times. Domains may be collections that the program computes, so we measure cost in $|\mathcal{A}|$ and d . For a fixed query and domains of size polynomial in $|\mathcal{A}|$, this is polynomial, so evaluation is in $FP^{\mathcal{O}}$, and it runs in polynomial time whenever every callable does. In combined complexity, nesting callables can make values grow exponentially, hence the condition on value sizes in footnote *b*. Termination of arbitrary callables cannot be guaranteed (Section 2).

In practice, most EQL queries involve at least one predicate or symbolic function, so the last row is the operative bound.

References

- [1] F. Baader, D. Calvanese, D. McGuinness, D. Nardi, and P. Patel-Schneider, editors. *The Description Logic Handbook: Theory, Implementation, and Applications*. Cambridge University Press, 2003.
- [2] S. Abiteboul, R. Hull, and V. Vianu. *Foundations of Databases*. Addison-Wesley, 1995.
- [3] A. K. Chandra and P. M. Merlin. Optimal implementation of conjunctive queries in relational databases. In *Proc. 9th Annual ACM Symposium on Theory of Computing (STOC)*, pages 77–90. ACM, 1977.
- [4] K. L. Clark. Negation as failure. In H. Gallaire and J. Minker, editors, *Logic and Data Bases*, pages 293–322. Plenum Press, 1978.
- [5] A. C. Klug. Equivalence of relational algebra and relational calculus query languages having aggregate functions. *Journal of the ACM*, 29(3):699–717, 1982.
- [6] R. Reiter. On closed world data bases. In H. Gallaire and J. Minker, editors, *Logic and Data Bases*, pages 55–76. Plenum Press, 1978.
- [7] N. Immerman. Languages that capture complexity classes. *SIAM Journal on Computing*, 16(4):760–778, 1987.
- [8] D. A. Mix Barrington, N. Immerman, and H. Straubing. On uniformity within NC¹. *Journal of Computer and System Sciences*, 41(3):274–306, 1990.
- [9] L. Hella, L. Libkin, J. Nurmonen, and L. Wong. Logics with aggregate operators. *Journal of the ACM*, 48(4):880–907, 2001.
- [10] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953.
- [11] B. Russell. On denoting. *Mind*, 14(56):479–493, 1905.
- [12] L. Libkin. Expressive power of SQL. *Theoretical Computer Science*, 296(3):379–404, 2003.
- [13] M. Y. Vardi. The complexity of relational query languages. In *Proc. 14th Annual ACM Symposium on Theory of Computing (STOC)*, pages 137–146. ACM, 1982.
- [14] B. Motik, B. Cuenca Grau, I. Horrocks, Z. Wu, A. Fokoue, and C. Lutz. *OWL 2 Web Ontology Language Profiles (Second Edition)*. W3C Recommendation, 2012.